PennyPal — Viva & Presentation Script

TechWiz 7: The World Tech Championship
Center: Aptech North Nazimabad Center
Theme: NextGen BudgetBee | **Category:** Multi-Platform App Computing
Lead Presenter: Muhammad Ammad
Project: PennyPal — Make Every Penny Count

Executive Summary & Setup

- Target Duration: 5 to 7 Minutes (followed by Judge Q&A)
- Representing Center: Aptech North Nazimabad Center
- Student Login: ammad.anees99@gmail.com / Ammad1234
- Admin Login: admin@pennypal.com / Admin@1234
- Core Architecture: Flutter 3.24+, Riverpod 2.0 (StreamProviders & Notifiers), Firebase Firestore (Offline-First Cache), Google Gemini Vision OCR
- Value-Added Features Beyond SRS: Split Bill Engine, Borrow & Lend (IOU) Tracker, Subscriptions Manager

---

Section 1: Self & Team Introduction (30 Seconds)

Spoken Script:

"Respected judges and panel members, good morning [or good afternoon].

My name is Muhammad Ammad. Together with my team, we proudly represent Aptech North Nazimabad Center in TechWiz 7 under the Multi-Platform App Computing category.

It is an honor for our team to be selected among the Top 12 in the world.

Today, we are thrilled to present our project: PennyPal — Make Every Penny Count."

Speaker Tips:

- Look straight into the camera with confidence.
- Clearly state your name and Aptech North Nazimabad Center.

Section 2: Why We Chose PennyPal Among the 3 Projects (30 Seconds)

Spoken Script:

"Among the three project tracks provided by the TechWiz committee, our team deliberately chose PennyPal because student personal finance is a universal everyday problem that we personally live every day.

Students receive pocket money, scholarships, or part-time stipends in cash, bank transfers, and mobile wallets. Traditional finance apps fail because they require paid subscriptions, complex accounting, and constant internet.

We chose PennyPal to build a truly student-first, offline-ready application that solves real student cashflow challenges."

Speaker Tips:

- Judges already wrote the problem statement, so do not recite the requirements back to them.
- Instead, highlight why your team picked this specific track and your personal passion for it.

Section 3: Deep Technical Architecture & Widget Hierarchy (1 Minute)

Spoken Script:

"Before moving to the live demo, let me walk you through our technical architecture:

1. Architecture Pattern: We followed Feature-First Clean Architecture. Each feature is cleanly decoupled into three distinct layers:

- Presentation Layer: UI screens, sheets, and reusable Riverpod consumer widgets.
- Domain Layer: Pure Dart data models, value objects, and business calculators with zero Flutter UI dependencies.
- Data Layer: Repositories and data sources handling cloud and local persistence.

2. State Management with Riverpod 2.0: We rejected standard setState and legacy Provider in favor of Riverpod 2.0:

- Compile-Time Safety: Riverpod eliminates ProviderNotFound runtime crashes completely.
- Decoupled from BuildContext: Our controllers and business logic do not rely on Flutter's widget tree context.
- StreamProvider & AsyncValue: We use StreamProviders to listen to Firestore real-time collections. AsyncValue cleanly handles loading, data, and error states without messy boilerplate.
- ConsumerWidgets & ConsumerStatefulWidgets: Only the specific widgets listening to a state slice rebuild on data changes, maintaining 60 to 120 FPS performance.

3. Offline-First Data Pipeline: Our Firestore implementation runs on local cache persistence. When a student adds an expense inside a university basement without internet, the transaction commits immediately to local SQLite/cache with optimistic UI updating. When connectivity returns, Firestore automatically syncs to the cloud in the background.

4. Machine Learning & Dual-Engine OCR: We integrated Google Gemini Vision 1.5 Flash for intelligent receipt parsing. To ensure absolute reliability, we built a fallback heuristic regex parser: if the AI API

times out, the app extracts the merchant, amount, and date offline without failing."

Speaker Tips:

- Judges love deep technical terms like Clean Architecture, AsyncValue, StreamProvider, and Optimistic UI updates.
- This proves you understand the entire codebase under the hood.

Section 4: Live Demonstration Walkthrough (3 to 4 Minutes)

(Share your screen. Speak each point clearly, then perform the exact action on your device or emulator.)

Step A: Secure Authentication & Route Guards

Screen Action: Launch PennyPal. Enter ammad.anees99@gmail.com and password Ammad1234. Tap Sign In.

Spoken Script:

"We begin with secure student authentication. Our routing is built with GoRouter and features declarative route guards. It listens to our authStateChanges stream: unauthenticated users cannot bypass the login screen, while authenticated students are routed directly to the dashboard, and admins to the admin portal."

Step B: The Student Dashboard & Real-Time Overview

Screen Action: Point to Net Balance, Income card, Expense card, and the Monthly Budget Ring chart.

Spoken Script:

"This is the Student Dashboard. It displays the student's Total Monthly Income, Total Expenses, and Net Balance aggregated across all accounts: Cash, Cards, and Mobile Wallets. Notice the animated Budget Ring: it calculates spending velocity in real-time. All numbers are powered by our pure Dart FinanceCalculator, which is completely decoupled from the UI for unit testing."

Step C: Calendar-Month Budgeting & 1-Tap Quick Copy

Screen Action: Tap the Budget tab. Show current month limits. Tap next month arrow (>) and show the 1-Tap Copy from Previous Month action banner.

Spoken Script:

"Budgeting in PennyPal is strictly calendar-month based. Student allowances, tuition fees, and hostel rent all follow calendar cycles, which makes historical month-on-month comparison accurate. Students set total limits and category-wise thresholds with proactive overspending badges. When a new month begins, students do

not need to re-type limits: our 1-Tap Quick Copy feature automatically inspects last month's budget and pre-fills all categories with one tap."

Step D: Expense Logging & AI Receipt Scanner

Screen Action: Tap Add Expense (+). Highlight the 8 mandatory categories. Select Food or Transport. Tap Attach Receipt to demonstrate camera and gallery attachment.

Spoken Script:

"Students can log transactions across all 8 mandatory SRS categories: Food, Transport, Education, Shopping, Entertainment, Bills, Savings, and Miscellaneous. We integrated an AI Receipt Scanner using Google Gemini Vision. It analyzes receipt photos to extract the vendor name, date, and amount. If network connectivity fails, our local heuristic fallback extracts the monetary data on-device so the student never experiences a crash."

Step E: Live Offline-First Demonstration (Key Evaluation Highlight)

Screen Action:

1. Pull down quick settings and turn on Airplane Mode (device is offline).
2. Log an expense of $15 for 'Campus Lunch'. Tap Save. Show that it appears immediately in the activity list.
3. Turn off Airplane Mode. Point out that Cloud Firestore synchronizes silently in the background.

Spoken Script:

"Here is our Offline-First architecture in real-time. I am enabling Airplane Mode — the phone is completely disconnected. I add a 15-dollar expense... as you can see, the transaction appears immediately on screen. Riverpod updates the UI optimistically while saving locally. Now, I turn off Airplane Mode... Firestore automatically detects connectivity and pushes the update to the cloud in the background."

Step F: Savings Goals & Milestone Calculations

Screen Action: Tap the Savings tab. Open an active goal (e.g., 'New Laptop'). Tap Contribute and add $20. Show the progress bar advance.

Spoken Script:

"In the Savings Goals module, students set target amounts, deadline dates, and monthly contribution commitments. PennyPal calculates remaining balance, progress percentage, and estimated completion time. When milestones are reached, the app celebrates progress to encourage continuous savings discipline. Completed goals are archived in goal history."

Step G: Value-Added Innovations Beyond the SRS

Screen Action: From dashboard shortcuts, showcase Split Bill, Borrow & Lend, and Subscriptions.

Spoken Script:

"SRS Section 1.9 allows teams to apply creativity beyond baseline specifications. Based on our campus user research, we engineered three major innovations:

1. Split Bill Engine: Students constantly split canteen meals, cab fares, and project printing. Our Split Bill sheet calculates individual shares with rounding protection and generates a pre-formatted payment reminder ready to send to WhatsApp in one tap.
2. Borrow & Lend Tracker (IOU): Peer borrowing is an everyday student reality. This tracker records who owes what, sets repayment due dates, and marks settlements so small informal loans never turn into lost money.
3. Subscriptions Manager: Students often lose track of recurring micro-charges like Spotify, gym passes, or cloud storage. This module tracks renewal dates and calculates their exact monthly burn rate impact."

Step H: Financial Literacy Hub & AI Assistant

Screen Action: Tap Learning, open a lesson card (e.g., 'Needs vs. Wants'). Then open the Assistant tab and demonstrate an AI query.

Spoken Script:

"Financial literacy is central to PennyPal. Our Learning Hub features structured lessons on budgeting, needs versus wants, and emergency savings. Students can also chat with our PennyPal AI Assistant for personalized, real-time advice on reducing discretionary spending."

Step I: Preconfigured Administrator Portal

Screen Action: Tap Profile, tap Logout. Sign in as admin@pennypal.com / Admin@1234. Show the Admin Dashboard.

Spoken Script:

"In accordance with SRS Section 1.6, we implemented a complete Preconfigured Administrator Portal. Logging in as Admin provides:

1. Student User Directory to monitor registered students.
2. System Analytics to inspect platform usage.
3. Feedback Moderation to review student ratings.
4. Support Helpdesk to resolve user tickets.
5. Curriculum CMS to publish and update learning lessons in real-time."

Section 5: Closing Statement & Handover (20 Seconds)

Spoken Script:

"In conclusion, PennyPal is not just an expense tracker. It is a production-grade, offline-first financial co-pilot engineered to build lasting financial habits for students.

On behalf of Aptech North Nazimabad Center, we thank you, respected judges, for your time and guidance. We are now ready for your questions."

Section 6: Top 20 Technical Judge Questions & Model Answers

Q1: "Why did you implement Split Bill, Debt Tracker, and Subscriptions when they were not in the SRS baseline?"

Answer: "SRS Section 1.9 explicitly encourages applying creativity to improve the cross-platform application. Our research at Aptech showed that split canteen meals, forgotten peer debts, and recurring subscriptions are the three largest causes of student financial leakage. Adding these completed the real-world student financial workflow."

Q2: "How does Riverpod manage state compared to Provider or Bloc?"

Answer: "Riverpod is a complete rewrite of Provider that operates outside Flutter's widget tree. It provides compile-time safety with no runtime ProviderNotFoundExceptions. We use StreamProviders for real-time Firestore synchronization and StateNotifiers for form and filter state, allowing selective widget rebuilds without unnecessary re-renders."

Q3: "What is the exact difference between ref.watch() and ref.read() in your code?"

Answer: "We use ref.watch() inside build() methods to subscribe widgets to reactive state changes so the UI rebuilds when data updates. We use ref.read() strictly inside event handlers and callbacks, such as button onTap functions, to read a provider once or trigger a method without creating an unwanted rebuild subscription."

Q4: "What happens if a student sets a budget mid-month on October 20th?"

Answer: "The budget document is keyed deterministically by month as YYYY-MM (e.g., 2026-10). All transactions recorded from October 1st to 20th are immediately aggregated against the new budget limit, updating remaining funds and triggering overspending warnings if past spending already exceeded the limit."

Q5: "How does receipt OCR work if Gemini API is offline or times out?"

Answer: "We engineered a Dual-Engine architecture in ReceiptOcrService. When an image is scanned, it queries the Gemini Vision API. If the call times out after 12 seconds or fails with a network exception, it seamlessly cascades to our local heuristic regex parser, ensuring zero downtime for the user."

Q6: "How is student data secured in Firestore?"

Answer: "We implement multi-tenant security rules in firestore.rules. Every document path is scoped under users/{userId}. Our rules enforce that request.auth.uid matches the document userId. No student can read or

write another student's ledger. Furthermore, sensitive CVV and full card numbers are never stored."

Q7: "How do you prevent negative balances or warn students of overdrafts?"

Answer: "Our FinanceCalculator computes live running balances per payment mode. If an expense exceeds the balance in a specific mode, such as spending $50 from a cash wallet holding $20, the app flags an overdrawn warning and updates the Financial Health Index into the 'Needs Attention' tier."

Q8: "How is the Financial Health Score calculated?"

Answer: "It is a 100-point algorithm divided into three pillars:

1. Budget Discipline (40 points): Compares actual spend against expected burn rate using a dynamic time-decay pace ratio (daysElapsed / totalDays).
2. Savings Margin (30 points): Evaluates whether savings exceed 25% of income.
3. Habit Consistency (30 points): Rewards daily transaction logging regularity and active savings goals."

Q9: "How does offline synchronization prevent data corruption?"

Answer: "Firestore provides client-side offline persistence. Local writes receive optimistic timestamps and a hasPendingWrites flag. When the connection re-establishes, Firestore synchronizes changes via atomic transactions. In the event of conflicting edits, Firestore's last-write-wins resolution guarantees document integrity."

Q10: "Why limit payment cards to two?"

Answer: "This is an intentional behavioral finance design choice. Research indicates that students juggling multiple credit and debit cards face higher rates of impulse spending and missed payments. Restricting to two cards encourages disciplined primary and emergency card usage and keeps the UI clean."

Q11: "How does PennyPal handle push and in-app notifications?"

Answer: "We use flutter_local_notifications paired with domain notification triggers. When a user logs an expense that crosses 80% or 100% of their monthly category budget, or when a subscription renewal or debt due date is within 48 hours, the app dispatches an immediate local notification."

Q12: "How do you optimize image handling for receipt attachments to prevent memory leaks?"

Answer: "Receipt photos are captured via image_picker and compressed before uploading. On the UI side, images are loaded using memory caching with fixed decode dimensions (cacheWidth and cacheHeight), preventing high-resolution camera images from consuming excessive device RAM."

Q13: "How does the app scale if a student logs thousands of transactions over four years?"

Answer: "Our queries are partitioned by calendar month and indexed with compound Firestore indexes on (userId, date). The app only queries the active month's transactions for the dashboard, using pagination and date-range limits on the Activity screen so data payloads remain tiny regardless of total history size."

Q14: "What automated tests did you write to ensure financial math accuracy?"

Answer: "We wrote automated Dart unit tests covering our domain calculation engines: FinanceCalculator, ReportCalculator, the budget pace ratio formula, and debt split rounding. Because our domain layer contains pure Dart models with zero Flutter UI dependencies, tests execute in milliseconds with 100% determinism."

Q15: "How does the app protect against SQL or NoSQL injection?"

Answer: "PennyPal uses the official Cloud Firestore SDK, which structures all operations as parameterized objects and typed document references rather than raw query strings. User inputs in text fields are strictly data payload values, making injection attacks impossible."

Q16: "What happens if a student changes their device clock to bypass a budget or goal deadline?"

Answer: "While local dates assist with offline entry, all cloud persistence uses Firestore's server-assigned timestamps (FieldValue.serverTimestamp()). When data synchronizes with the backend, cloud validation guarantees audit accuracy regardless of client clock tampering."

Q17: "How did you ensure responsive UI across phones, foldables, and tablets?"

Answer: "We built an AdaptiveScaffold and custom responsive breakpoints. On standard mobile screens, navigation uses a bottom navigation bar. On tablets and desktop web browsers, the layout automatically transforms into a persistent navigation rail with multi-column grid cards, maximizing screen real estate."

Q18: "Can students export reports for parents or university scholarship audits?"

Answer: "Yes. From the Reports module, students can generate comprehensive monthly statements and export their full ledger to CSV or formatted print-ready PDF files using Flutter's printing and pdf packages."

Q19: "How do you support international currencies for students studying abroad?"

Answer: "Our CurrencyService allows students to select their local currency symbol (such as PKR, USD, EUR, GBP, or INR). All financial formatting in the UI passes through our centralized CurrencyFormatter, ensuring seamless formatting across all screens."

Q20: "What is your roadmap for monetization or scaling PennyPal beyond TechWiz?"

Answer: "Our roadmap follows a B2B2C model: licensing PennyPal directly to universities and colleges as an official campus wellness tool, integrating student meal cards, and offering curated affiliate discounts on educational supplies and textbooks."

Section 7: Professional Viva Delivery Guidelines

Clarification Phrases (If you need the judge to repeat):

- If audio is unclear: "Pardon me sir/ma'am, could you please repeat the question?"
- If the question is fast: "Could you please elaborate on that point, sir?"
- To verify the question before answering: "Are you inquiring about our offline synchronization mechanism, or our Riverpod state flow?"

- To demonstrate proof on screen: "Yes sir, let me demonstrate that live on screen right now."

Key Presentation Rules:

1. Speak in solid, clear sentences. Never rush.
2. When answering technical questions, name the specific file or class (e.g., FinanceCalculator, ReceiptOcrService, StreamProvider).
3. Always keep your emulator or device running with live test data loaded before the meeting starts.

Good luck Team PennyPal (Aptech North Nazimabad Center)